

Exploratory Testing

ENGI- 9839



Today's Goals

- Exploratory Testing

Exploratory Testing

- Tester write down ideas to give direction, then create tests “live”.
 - Tester chooses next action based on results seen.
- Can find subtle faults missed by formal testing.
 - Allows tester to better learn system functionality, and identify new ways of using features.

Session-Based Exploratory Testing

- Time-based method to structure exploratory testing.
 - Conducted with no e-mail, phone, messaging.
 - Short (60min), Normal (90m), Long (120m)
- Primary components:
 - **Mission**
 - The purpose of the session.
 - Provides focus for the tester.
 - **Charter**
 - Individual testing goals to be completed in this session.
 - Could be a list of features or scenarios.

Session Report Items

- **Mission: Overall goal**
 - “Analyze Login Feature on Website”
- **Charter: Features and scenarios to focus on.**
 - “Login as existing user with username and password”
 - “Login as existing user with Google account”
 - “Login as existing user with Facebook account”
 - “Enter incorrect username and password to verify validation message”
 - “Block your username and verify the validation message”
 - “Use Forgot Password link to reset password”

Session Report Items

- **Start and end time of session**
- **Duration of session**
- **Testing notes:** journal of actions taken
 - Opened login page
 - Verified default screen.
 - Verified that existing and new user account links exist.
 - Opened existing user login
 - Verified successful login with username, Google, and Facebook.
 - Verified validation messages.

Session Report Items

- **Fault Information:** Describe each fault. File a bug report, include tracker ID.
- **Issues Information:** If an issue prevents or complicates testing, describe it.
 - Include **data files** (screenshots, recordings, files).
- **Set-up Time:** % of time required to set-up.
- **Test Design and Execution Time:** % of time spent purely on testing

Session Debrief

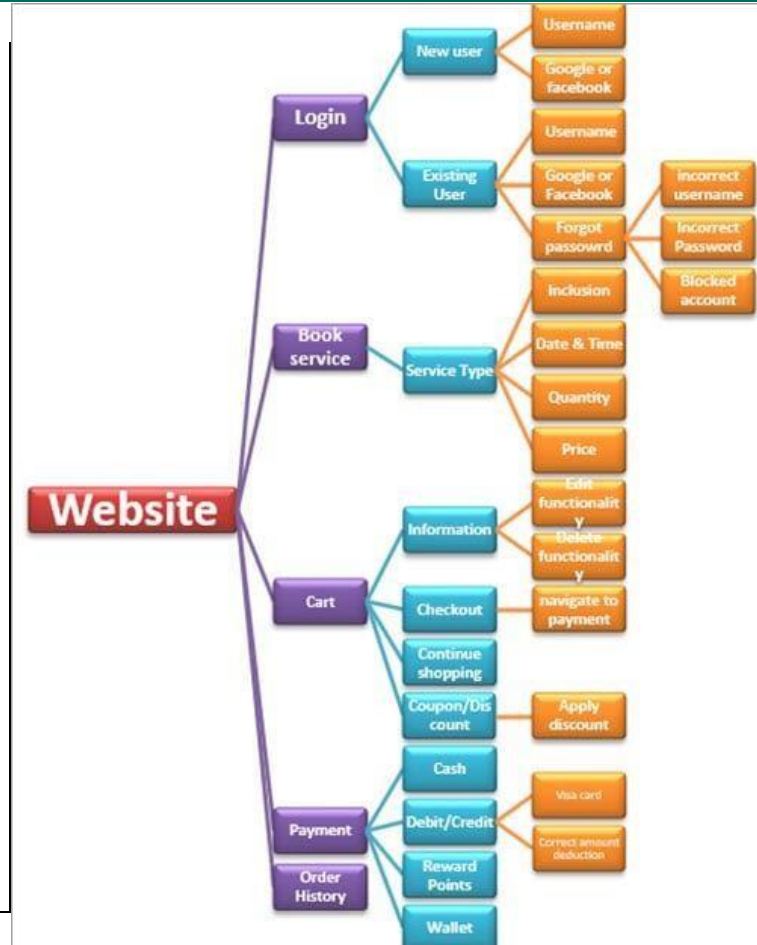
- Short meeting between tester and manager to review the findings.
- Track time spent testing, number of faults reported, time spent on set-up, time spent on testing, time spent analyzing issues, features covered.
- Allows time management and process observability.

Tips for Exploratory Testing

- Divide the application into modules or features, then try to further divide.
- Make a checklist of all the features and put a check mark when each is covered.
- Start with a basic scenario and then gradually enhance it to add more features to test it.

Example

- Start with functionality you know well (Login)
- Examine possible options and list them.
- Use your findings to plan the next steps.
- As you learn and observe, more test cases will emerge.

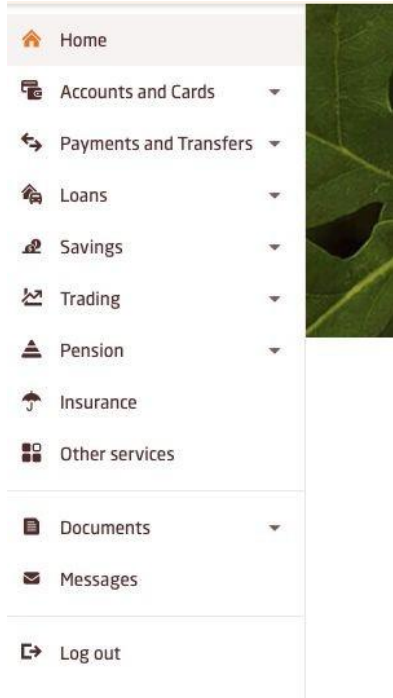


Tips for Exploratory Testing

- Test all input fields.
- Check for all possible error messages.
- Test all negative scenarios.
 - Invalid input, mistakes in usage.
- Check the GUI against standards.
- Check the integration of the application with other external applications.
- Check for complex business logic.
- Try to do the ethical hacking of the application.

Example System

RBC Bank



- Banking Webapp
- **How would you perform exploratory testing?**
 - Scenarios you would try?
 - Features you would focus on?

Tours in Exploratory Testing

Using “Tours” in Exploratory Testing

- A tourist seeks to visit as many districts of a city as possible within the time budget.
 - In software, the “city” is the system, and the “districts” are aspects of the system.
- A **tour** is a plan for exploratory testing.
 - Includes a set of objectives, based on visiting different “districts”, to focus on during testing.
 - Should take less than four hours.

Exploratory Tours

- Features split into “districts” based on type and how we test.
 - Business = core functionality
 - Seedy = security aspects
- “Tours” related to each district.
 - Each prescribes a way of exploring the software.

EXPLORATORY TESTING TOURS

- 1 Business District**
 - Guidebook tour
 - Money tour
 - Landmark tour
 - Intellectual tour
 - FedEx tour
 - After-Hours tour
 - Garbage Collector tour
- 2 Historic District**
 - Bad-Neighborhood tour
 - Museum tour
 - Prior Version tour
- 3 Entertainment District**
 - Supporting Actor
 - Back Alley
 - All-Nighter or Clubbing tour
- 4 Tourist District**
 - Collector tour
 - Lonely Businessman
 - Supermodel
 - Scottish Pub tour
- 5 Hotel District**
 - Rained-Out tour
 - Couch Potato tour
- 6 Seedy District**
 - Saboteur tour
 - Obsessive-Compulsive tour
 - Antisocial tour

Business District

- Most important features.
 - The functionality that will get users to buy software.
- Tours focus on features that are used most often.
 - **Guidebook Tour:** Focuses on common user journeys, covered in user manuals and tutorials.
 - **Fed-Ex Tour:** Focuses on how data is passed and transformed between these features.



Guidebook Tour

- Cities advertise top attractions, and ensure they are clean and safe.
- Software offers user manuals and tutorials, illustrating step-by-step use of features.
 - Follow tutorials and execute each step.
 - Tests both functionality and accuracy of tutorials.
 - If the app doesn't behave exactly like the documentation promises, you've found a bug.



Guidebook Variants

- “Pundit’s Tour”
 - Create tests based on complaints.
 - Try to reproduce their issues.
- “Competitor’s Tour”
 - Perform tour on competing products and their guides.
 - Identify potential improvements to your system.

Fed-Ex Tour

- When a package is sent, it is handled by many people and passes through many locations.
 - In software, data is passed, transformed, and passed again before output appears.
- Examine how data is manipulated.
 - Validate data after operations.
 - You are testing the transit system of the data.



Historic District

- Historic districts contain important old buildings.
- In software, these are older features still in use.
- Tours verify that they still work and are fault-free.
 - **Bad Neighborhood Tour:** Ensure that faulty code now works, and that fixes did not introduce new faults.
 - **Museum Tour:** Ensure that unchanged code still works as intended.



Bad Neighborhood Tour

- Complex features may have had many faults fixed over time.
- Focus on those features and ensure that:
 - Reported faults have actually been fixed.
 - New faults have not been introduced or uncovered.
- Also check related features for introduced faults.



Museum Tour

- Older features may not have been modified or retested recently.
- Verify that old code still works in the current system.
 - Check modification dates in repository, and ensure oldest elements are retested.
 - Such elements often lack tests, are hard to modify, not tested up to current standards.



Entertainment District

- Entertainment districts fill in the gaps in a vacation when you want to relax.
 - In software, this represents supporting features that aren't part of critical functionality.
 - Word processor: Making document look nice.
- Tours visit supporting features and ensures they are properly intertwined with core features.
 - **Supporting Actor:** Features on-screen with core features
 - **All-Nighter Tour:** Run the software for a long time.

Supporting Actor Tour

- Many features might be linked to a core feature.
 - When we search for a product (core feature), we see “reviews” and “similar items” (non-core features).
- Focus testing on features that share the screen with core features.
 - Will be used often.
 - Make sure they can be accessed from the core feature.



Tourist District

- Captures the experience of being a tourist - visit functions quickly and avoid deep inspection of individual features.
- **Souvenir Tour:** Run quick tests on functions, examine actions and identify gaps, plan round 2.
- **Supermodel Tour:** Test the GUI thoroughly, look for GUI errors, inconsistencies, usability errors.

Supermodel Tour

- Ignore the functionality and focus on the GUI.
- As you try different functions:
 - Does GUI render properly and quickly?
 - Are transitions clean?
 - Are colors and styles used consistently?
 - Is GUI usable and accessible by those with dyslexia or colorblindness?



Supermodel Tour Example

- **Dynamics AX Client**

- Resource planning system acquired by Microsoft.
- Shift from APIs to heavy GUI development.
- Led to take-up of exploratory testing.
 - Found MANY bugs missed by API tests.
 - Many new scenarios and interactions not considered before.
 - Testers learned that they knew very little about their own app.
 - Now: exploratory testing before new features merged.

Hotel District

- Return to hotel to take a break.
- Focuses on secondary and supporting usage scenarios.
 - Software “at rest” can be very busy.
 - **Rained Out Tour:** Cancel running operations and see if problems are caused.
 - **Couch Potato Tour:** Leave fields blank and use default values to assess ability to process partial information.



Rained-Out Tour

- Look for operations that can be cancelled.
 - Cancel midway through, see if everything still works.
- Good for finding failures related to the program's inability to clean up after itself.
 - Open files, corrupted memory or state.
- Even if there is no cancel button, can click back button or close entirely.



Rained-Out Tour Example

- **DynamicsAX**
- Change the state of the software before cancelling.
 - Opened a pop-up within a form, then closed the form while pop-up was open.
 - App crashed because pop-up was still open.

Couch Potato Tour

- Tester does least interaction possible.
 - Leave default values in place, leave input fields blank, try to move forward without offering much data.
- Ensures software must execute code for processing blank or partial information and defaults.
 - We try so many complicated scenarios that we can miss or forget the defaults.



Seedy District

- Focused on attacking and breaking the system.
 - **Saboteur Tour:** Directly attack software via malformed input or resource manipulation.
 - **Antisocial Tour:** Try unlikely input or perform actions in the wrong order.
 - (add 10000 songs, try to play empty playlist, order 10000000 pairs of shoes)



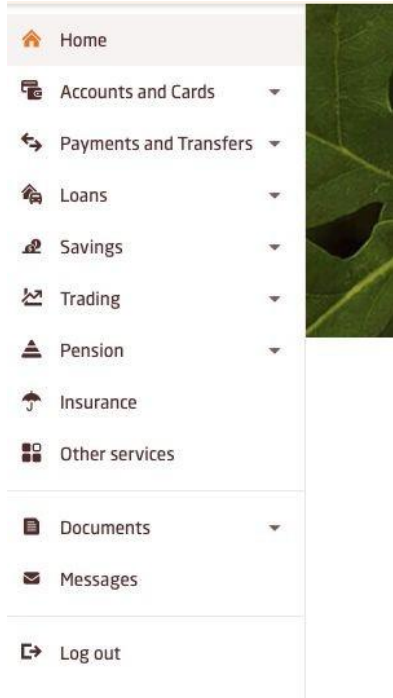
Saboteur Tour



- Force the software to act.
- Understand the resources it requires to successfully act.
- Remove or restrict those resources.
 - Use corrupt input data, limit network connectivity, allow too little RAM, run many other apps at the same time.
- Think of ways to creatively disrupt operations and try them out.

Revisiting the Example System

RBC Bank



- Banking Webapp
- **How would you perform exploratory testing?**
 - Scenarios you would try?
 - Features you would focus on?
 - Particular tours?

We Have Learned

- Exploratory Testing
 - Tests are not created in advance.
 - Testers check the system on-the-fly,
 - Often based on ideas noted before beginning.
 - Testing as a thinking idea.
 - About discovery, investigation, and role-playing.
 - Test design and execution done concurrently.
 - Often by directly using the software and its user interfaces.

We Have Learned

- Tours apply different focus areas to exploration
 - Business District: Core features
 - Historic District: Legacy code and old software versions
 - Entertainment District: Supporting functionality, long execution sessions
 - Tourist District: Looks for gaps in the experience, iterative fast rounds of exploration.
 - Hotel District: Focuses on supporting functionality
 - Seedy District: Attacks and misuse of software

Next Time

Example of Exploratory testing